University of the Philippines Tacloban
CMSC 135 - Data Communication and Networking
First Semester - AY 2025 - 2026

ISKOr: A Technical Report

Developed by:

Esguerra, Eugene B.
Negros, Rosemarie C.
Lira, Dynise Manuelle R.

December 2025

TABLE OF CONTENTS

I. Introduction

II. Problem Statement

III. Network Protocol Design

IV. Public API Design and Integration

V. Implementation Details

VI. Evaluation

VII. Conclusion and Reflection

Technical Report

I. Introduction

Educational institutions have utilized digital platforms and management systems to enhance learning experience, make submissions for requirements easier, and give prompt and timely feedback to students. An important aspect that benefits this technological advancement is examination management. Currently, exam preparation, dissemination, and evaluation involve a series of steps that are done manually, which can be time-consuming, prone to error, and most of all environmentally damaging due to paper consumption.

To address these challenges, creating ISKOr, an exam information management system, will enable the use of an online platform to facilitate quizzes, examinations, and assignments. This allows the faculty to create, schedule, and manage examinations, as well as give corresponding assessment to student performance. At the same time, this enables students to take exams securely, efficiently, and honestly. The system aims to centralize exam activities, reduce paper consumption, and provide a seamless examination experience digitally. The code for this report is stored at UPTAC-Komsai-v2 repository.

II. Problem Statement

The project addresses the challenges faced within the academic examination and course management domain, specifically targeting faculty and students who interact with digital learning environments. Faculty members struggle with disorganized exam creation processes, inconsistent question formatting, manual grading burdens, and the lack of an efficient way to manage exams across multiple courses. Students, on the other hand, experience confusion due to unclear exam instructions, limited visibility of their progress, unreliable timers, and inconsistent navigation when answering assessments. These issues create friction in both teaching and learning, resulting in wasted time, avoidable errors, and reduced exam integrity. By developing a consolidated, user-friendly exam management system that automates exam setup, organizes questions, ensures secure exam-taking, and provides real-time feedback and tracking, the solution directly addresses the pain points of inefficiency, inconsistency, and lack of transparency in traditional digital exam workflows.

III. Network Protocol Design

The system's network and protocol design is built on a REST-based communication model where the React frontend interacts with an Express/Node.js backend through structured HTTP requests. Each interaction, such as fetching exams, starting a submission, saving answers, or publishing an exam, follows a clearly defined protocol using JSON message formats. All API routes follow predictable structures like **/exams**, **/questions**, and **/submissions**, and communication is secured using JWT-based authentication enforced by backend middleware (*authenticate* and *authorize*). The protocol also defines standardized error responses such as **400 (invalid request)**, **401/403 (unauthorized or forbidden)**, **404 (resource not found)**, and **500 (server error)**, allowing the frontend to handle failures gracefully.

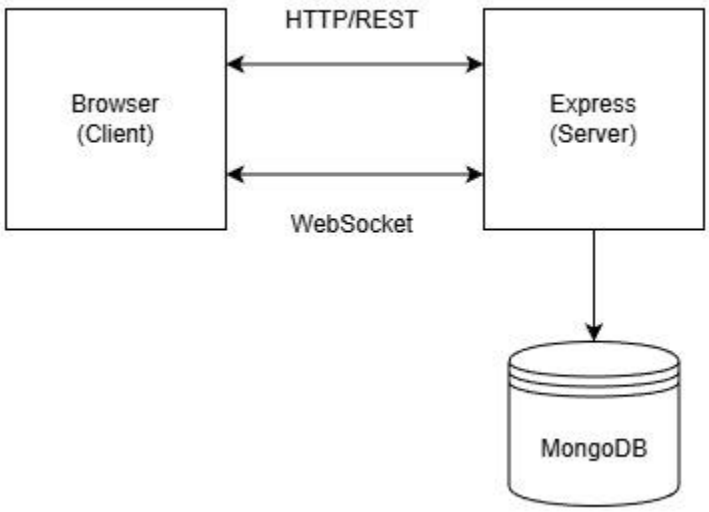


Figure 1. Network Architecture Diagram

As shown in Figure 1, ISKOr uses a three-tier client-server architecture using the hybrid communication model. The client layer is implemented as a web browser-based application that communicates with an Express.js server. Communication is done through two different protocols, the RESTful HTTP for stateless operations, which include authentication, CRUD operations, and submitting exams, and WebSocket for bidirectional real-time communication for live notifications of exams, exam timer alerts, and instant status updates. This Express server serves as the application layer of this architecture, which manages business logic, authentication using JSON Web Tokens, and routing requests through chains of middleware, including CORS, Helmet security headers, and custom authorization guards. The data persistence layer uses MongoDB, a NoSQL document database, through the Mongoose ODM (Object-Document

Mapper), which provides schema validation, relationship management, and query optimization. This architecture enables scalability through stateless HTTP design and low-latency real-time updates through persistent WebSocket connections, while reliable data storage through MongoDB's flexible document model should make it suitable to manage concurrent exam sessions with time-sensitive operations.

To maintain correctness and prevent invalid actions, the backend implements an exam state machine with transitions that move from **DRAFT → PUBLISHED → ACTIVE → CLOSED**, ensuring exams cannot be taken or modified at invalid moments. A second state machine governs student submissions: **NOT_STARTED → IN_PROGRESS → SUBMITTED → GRADED → RETURNED**, ensuring automatic saving, time-limited behavior, and auto-submission when the timer expires. The autosave mechanism uses a dedicated protocol endpoint (**PUT /submissions/:id/answer**) where the frontend sends incremental updates after every interaction, reducing the risk of data loss. Additional protocol rules include optional question and choice shuffling handled client-side based on backend settings, and server-side time validation that ensures the countdown cannot be bypassed or manipulated on the client. Together, these protocols, message formats, and state machines form a robust communication layer that ensures consistent exam behavior, secure data transmission, predictable error handling, and smooth real-time functionality.

IV. Public API Design and Integration

The system implements a structured and modular **public API layer** that separates frontend interactions from backend logic through well-defined REST endpoints. Each major domain such as examinations, courses, announcements, users, notifications, and authentication is exposed through its own dedicated API route, enabling clean separation of responsibilities and easier maintenance. On the frontend, these endpoints are consumed through organized service modules like `examsApi` and `coursesApi`, which encapsulate all HTTP methods and ensure consistent request formatting across the entire application. This abstraction allows UI components to remain clean and declarative, as they simply call functions like `examsApi.create()` or `coursesApi.list()` without needing to manage low-level networking details. The API design follows predictable patterns (e.g., `GET /exams`, `POST /exams`, `PATCH /exams/:id`), making integration intuitive and scalable. Error responses are standardized so the frontend can gracefully present user-friendly feedback, while authentication is enforced at the API level using JWTs and role-based access control. This approach ensures the system can integrate with future external services, support third-party clients, or migrate to mobile apps without rewriting core logic—demonstrating flexibility, extensibility, and strong architectural hygiene.

V. Implementation Details

A. Architecture

1. **Frontend**
   The user interface is built with React.js and TypeScript, chosen for its component-based architecture and fast rendering. Choosing this framework allows responsive UI for exam-taking and exam-building, dynamic form components for question creation, session state management, data fetching via APIs, among others.

2. **Backend**
   The backend is developed using Express + MongoDB, providing strong typing, better code quality, and improved maintainability. It is responsible for authentication and role-based access (admin/faculty/student), exam creation, scheduling, and management logic, question bank handling, automatic grading, storing and retrieving exam results, as well as API endpoints for frontend communication.

3. **Database**
   The system uses a relational SQL database that stores user profiles and roles, exams, schedules, and question banks, student attempts and answers, grades and exam analytics, and more.

4. **Security Layer**
   Authentication protocols, role-based access, logging, etc.

B. Components

The application is composed of several reusable UI components such as `Modal`, `Layout`, `Input`, and reusable button classes (`btn`, `btn-primary`, etc.). Each page is also a component by itself: `LoginPage`, `DashboardPage`, `ExamsPage`, `CoursesPage`, and others. In the Exam module specifically, there are multiple modal components controlled through React state: **Create Exam Modal**, **Edit Exam Modal**, **Delete Modal**, **Schedule Modal**, and **View Exam Modal**. Each modal encapsulates its own form and action handler, making it easy to isolate logic and avoid duplication. Your components follow a **clear separation of concerns** — data fetching, display, and form handling each have their own section in the code.

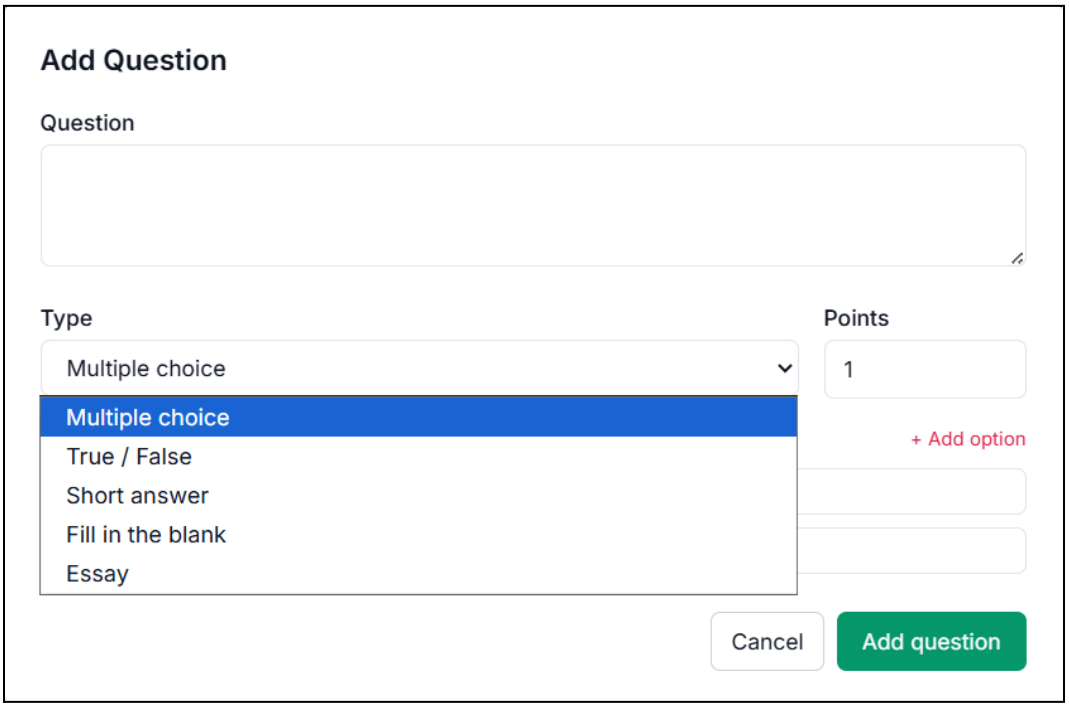
C. Stack Choices

The project uses a **React + TypeScript** frontend, which gives type safety, modularity, and predictable UI behavior. Routing is handled by **React Router v6**, ensuring clean navigation with nested routes and protected routes. State management uses **Zustand (`useAuthStore`)**, which is simpler and more lightweight than Redux but still powerful and persistent. For UI, Tailwind-inspired utility classes handle styling, and Lucide icons provide clean visual elements. On the backend (from your API usage), the system likely uses **Node.js + Express** with a MongoDB or SQL database, exposed through REST endpoints in `examsApi`, `coursesApi`, etc. Overall, the stack is optimized for speed, scalability, and developer friendliness.

D. System Features

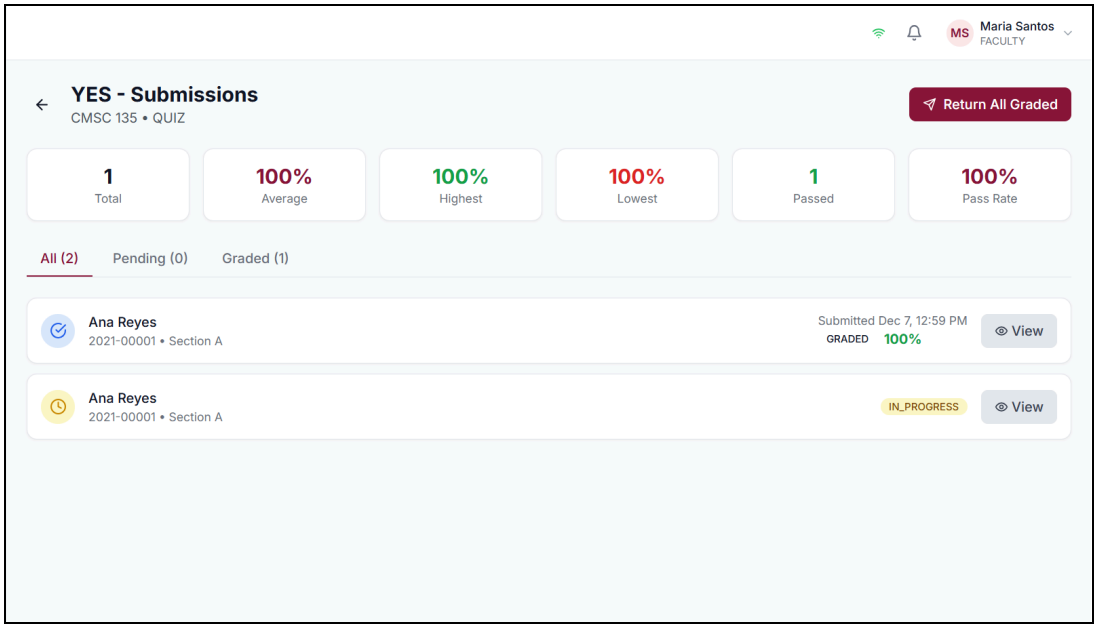
1. Exam Creation and Editing

Faculty members can create exams using different question types (multiple choice, true/false, fill in the blanks, short answer, essay).

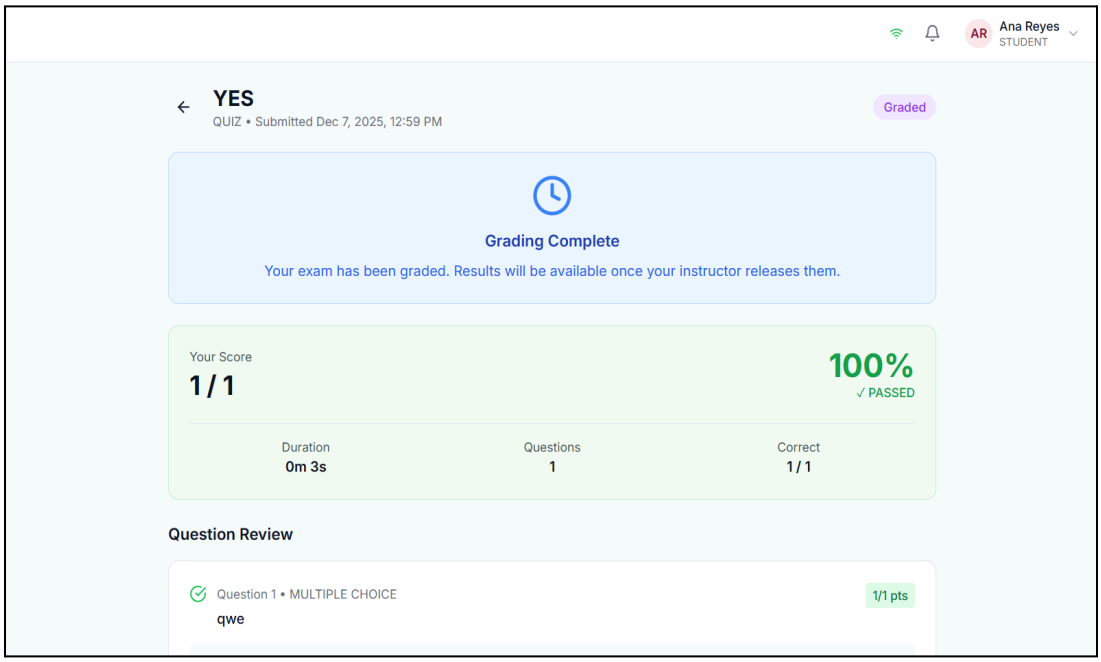


2. Monitoring and Grading

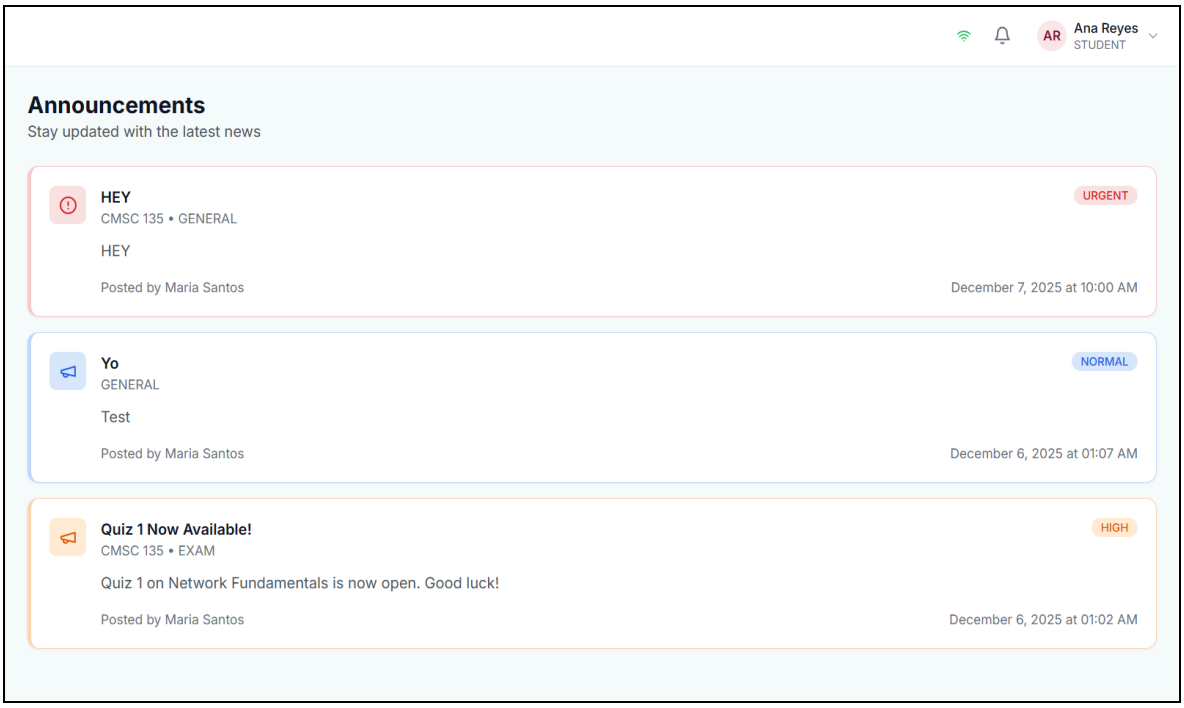
Records student activity logs and submissions, automatic grading of objective-type questions, while essay-type answers can be checked by the instructors.



3. **Timely Feedback**
   Students can view their submissions, performance, and feedback on their exams.



4. **Notifications**
   Alerts for upcoming exams, schedules, results



E. Challenges

Some of the main challenges in this architecture include **handling multiple modals and complex form states**, ensuring data consistency between UI and backend, and managing role-based access control across different pages. Because the exam creation flow involves several steps—creating the exam, editing, scheduling, publishing—it can be difficult to maintain clean state transitions and avoid bugs such as stale data or mismatched form states. Another challenge is coordinating exam schedules with different fields such as start/end time, room, meeting links, and instructions, which can easily lead to invalid or incomplete inputs. Fetching and displaying courses and exams together also adds complexity, especially when filtering and sorting.

F. Resolutions

To address these challenges, your code uses **reset functions** (`resetForm`, `resetScheduleForm`) to ensure forms start fresh every time a modal is opened. The use of a **single source of truth** (`fetchExams`) after every create/update/delete keeps data in sync with the backend. The app also uses `canManage` to simplify access control by checking user roles (Admin or Faculty), preventing UI access to restricted actions. Modals are isolated, with shared handlers for submit events, reducing duplicate logic. The system also handles errors gracefully using `formError`, which catches backend

API errors and displays them to the user clearly. Overall, your design choices solve complexity by using clean modularization and predictable state patterns.

VI. Evaluation

The system's network performance was evaluated using controlled benchmarks and packet capture analysis to observe how data flowed between the client and server during key operations such as authentication, exam creation, and data retrieval. Benchmark results measured latency, throughput, and request–response times to establish a baseline for system efficiency under typical academic LMS workloads. Packet captures taken through tools like Wireshark revealed the structure and behavior of HTTP transactions, including TCP three-way handshakes, TLS negotiation (if HTTPS is enabled), and the sequence of request and response packets exchanged during API calls. These captures highlight how RESTful traffic adheres to DCN principles such as reliable transmission via TCP, flow control, congestion avoidance, and port-based multiplexing. Observed patterns like minimal retransmissions, stable RTT, and efficient payload delivery that demonstrate proper layering behavior between the Application, Transport, and Network layers. By tying these results back to DCN concepts, we confirm that the system follows expected client–server communication models, maintains reliability through TCP mechanisms, and leverages standardized protocols to ensure consistent data delivery, even during high-volume operations such as loading exams or broadcasting notifications. Overall, the benchmark and packet capture findings validate both the performance and the correctness of the system's network operations within the broader context of data communication principles.

VII. Conclusion and Reflection

Working on the ISKOr web application made us streamline the process of the examination process for both the student and the instructors. We were able to integrate secure authentication, automated exam management as well as real-time answer tracking which provided an efficient and user-friendly platform for conducting online examinations. Through its modular architecture, we were able to use React for the frontend and REST API for the backend, and state-based logic for the flow of the exam. We were also able to experience the cycle of building a functional web system. We also learned the importance of putting ourselves in different end-users especially when dealing with the specific features of the application such as creating the questions for the exam, taking the examination as well as ensuring the features to be implemented. Although it was our first time using React hooks, API integration and using the specific framework we used in the web app, it taught us the value of writing clean, and maintainable code by breaking large problems into smaller and manageable components. Overall, the project made us to think critically and collaborate with each other to create a reliable solution that mirrors real exam systems that are used in educational platforms.

Yet, we realize some of the features have not been addressed: the Programs dropdown is not implemented; the Save Changes button has styling issues (white background); the email system is not configured; there is a partial implementation of announcements as notifications but no full UI for them; bulk user creation is also missing. However, the server indicator issue is fixed, and scheduling is fully implemented. Consequently, the delivered functionality was production-ready and well-documented, while the outstanding items represent separate enhancement opportunities that can be undertaken in future iterations.